



Otimização de Rotas

Disciplina: Programação Paralela

Tema: Aplicação de algoritmos de busca e OpenMP para cálculo eficiente de rotas de evacuação.



Motivação do Projeto

O problema

- Crescimento do tráfego urbano.
- Situações de emergência exigem decisões rápidas.
- Sistemas convencionais podem não responder em tempo hábil.

Exemplos de regiões críticas

- Anel Rodoviário
- Avenida do Contorno
- Avenida Cristiano Machado



Objetivo do Projeto

Objetivo

Desenvolver uma solução capaz de calcular múltiplas rotas de evacuação simultaneamente utilizando programação paralela.

Especificamente:

- Modelar a malha viária como um grafo.
- Utilizar algoritmos de caminhos mínimos.
- Paralelizar o processamento das consultas.
- Avaliar desempenho através de speedup e eficiência.

Modelagem do Problema

Representação do mapa

- **Vértices:** cruzamentos e interseções.
- **Arestas:** ruas e avenidas.
- **Peso das arestas:** tempo estimado de deslocamento.

A

/ \

4/ \2

/ \

B---3---C

\

5

\

D

O objetivo é encontrar o caminho de menor custo entre um ponto congestionado e uma saída da cidade.



Cenário de Estresse

Elemento		Quantidade
Vértices		200.000
Consultas simultâneas		4.000
Origens	Pontos congestionados	
Destinos	Saídas da cidade	
Base utilizada	Geofabrik	



Algoritmo de Busca

Dijkstra

Para cada origem congestionada:

1. Inicializa as distâncias.
2. Explora os vértices mais promissores.
3. Atualiza os menores caminhos encontrados.
4. Determina a saída mais próxima.

Pseudoestrutura:

Para cada origem:

Executar Dijkstra

Comparar distância até cada saída

Guardar a menor rota encontrada



Estratégia de Paralelização

Ideia principal

Cada cálculo é independente.

Origem 1 → Dijkstra

Origem 2 → Dijkstra

Origem 3 → Dijkstra

Origem 4 → Dijkstra

↓

Thread 0

Thread 1

Thread 2

Thread 3

Não existe dependência entre as tarefas.

Por isso, o problema é considerado **Embarrassingly Parallel**.



Uso do OpenMP

Trecho principal:

```
#pragma omp parallel for schedule(dynamic)
```

```
for (int i = 0; i < totalOrigens; i++) {
```

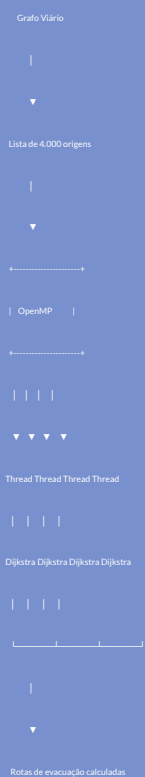
```
    auto distancias = dijkstra(origens[i]);
```

```
    calcularMelhorSaida(distancias);
```

```
}
```

Cada thread processa uma origem diferente, permitindo aproveitar múltiplos núcleos do processador.

Arquitetura da Solução





Resultados



Interpretação dos Resultados

- O aumento do número de threads reduz quase proporcionalmente o tempo de processamento.
- A eficiência permanece acima de 90%, indicando baixo overhead.
- A independência entre as tarefas favorece excelente escalabilidade.



Conclusão

Conclusões

- A programação paralela tornou o cálculo das rotas muito mais rápido.
- O uso de OpenMP simplificou a distribuição das tarefas entre as threads.
- O algoritmo de Dijkstra mostrou-se adequado para encontrar caminhos mínimos na malha viária.
- A estratégia de decomposição por tarefas apresentou excelente escalabilidade em cenários de grande porte.